Wi-Fi Deauthentication Attack Detector

1ST YEAR CYBERSECURITY & IOT PROJECT GUIDE

Level: Beginner / First-Year **Hardware:** ESP8266 (NodeMCU) **Domain:** Wireless Security & IoT

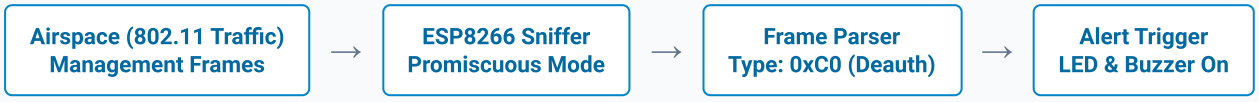
1. Project Executive Summary

A Wi-Fi Deauthentication (Deauth) attack targets IEEE 802.11 management frames, allowing an attacker to force client devices off a Wi-Fi network without knowing the password. This project constructs an embedded defense monitor using an ESP8266 microchip. Operating in promiscuous mode, the device intercepts raw 802.11 frames, isolates deauth management frames, and triggers physical alarms when suspicious thresholds are exceeded.

2. Bill of Materials & Hardware Setup

Component	Description	Connection Pin
ESP8266 NodeMCU	Wi-Fi SoC with promiscuous packet sniffing capacity	USB Power / Micro-USB
Red LED	Visual alert indicator for attack detection	Anode to GPIO 5 (D1), Cathode to GND
Active Buzzer	Audible alarm when deauth flood is active	Positive to GPIO 4 (D2), Negative to GND
220Ω Resistor	Current limiting resistor for the LED	In series with LED Anode

3. Detection Architecture & Dataflow



4. Environmental Setup

- **Software:** Download and install the latest version of Arduino IDE.
- **Board Manager:** Navigate to `File > Preferences` and add `http://arduino.esp8266.com/stable/package_esp8266com_index.json` to the Additional Boards Manager URLs.
- **Library Installation:** Open `Tools > Board > Boards Manager`, search for `esp8266`, and click Install.

5. Core Detector Firmware Code

Flash the following C++ code to the ESP8266 NodeMCU via the Arduino IDE:

```
#include <ESP8266WiFi.h> #define LED_PIN 5 // GPIO5 (D1) #define BUZZER_PIN 4 // GPIO4 (D2) unsigned int deauth_count = 0; unsigned long last_reset_time = 0; const int ATTACK_THRESHOLD = 5; // Frames per second threshold void sniffer(uint8_t *buf, uint16_t len) { if (len < 28) return; // Check for 802.11 Deauth Subtype: 0x00C0 if (buf[12] == 0xC0 || buf[12] == 0xA0) { deauth_count++; } } void setup() { Serial.begin(115200); pinMode(LED_PIN, OUTPUT); pinMode(BUZZER_PIN, OUTPUT); digitalWrite(LED_PIN, LOW); digitalWrite(BUZZER_PIN, LOW); WiFi.mode(WIFI_STA); wifi_promiscuous_enable(1); wifi_set_promiscuous_rx_cb(sniffer); Serial.println("Deauth Detector Active. Sniffing..."); } void loop() { if (millis() - last_reset_time >= 1000) { if (deauth_count >= ATTACK_THRESHOLD) { Serial.printf("ALERT: Deauth attack detected! Frames: %d/s\n", deauth_count); digitalWrite(LED_PIN, HIGH); digitalWrite(BUZZER_PIN, HIGH); } else { digitalWrite(LED_PIN, LOW); digitalWrite(BUZZER_PIN, LOW); } deauth_count = 0; last_reset_time = millis(); } }
```

6. Operational Testing Procedure

1. Select board **NodeMCU 1.0 (ESP-12E Module)** under `Tools > Board`.
2. Connect the ESP8266 to your PC, select the correct **COM Port**, and press **Upload**.
3. Open **Serial Monitor** at a baud rate of `115200`.
4. To test defensively: Use a second test micro-controller (or authorized tool) on a private lab network to send test deauth packets and confirm the LED and buzzer activate instantly.

> **Cybersecurity Concept & Defensive Relevance**
> Deauthentication frames are intentionally unencrypted in legacy WPA2/802.11 standards to permit rapid network disconnections. Modern defenses rely on **802.11w (Protected Management Frames)** or upgraded **WPA3** protocols to authenticate these management frames and prevent spoofing attacks.